Scope-Aligned Summary Tokens in a Hybrid-Attention Code Model: A Controlled Study with a Negative Result

ABSTRACT

Summary (or "gist") tokens compress a span of context into a few positions, so that later tokens can reach the span through them instead of through its raw key–value entries. For source code, a natural place for such tokens is the end of each syntactic scope (function, class, module run). We retrofit this idea onto the sliding-window layers of Gemma 4 E2B with LoRA and compare seven variants under matched token counts and, where it matters, matched key–value (KV) memory: no summaries, fixed-interval chunks, scope-aligned summaries, scope-local attention windows, call-graph edges between summaries, and memory- and count-matched versions of the scope and chunk variants. Across language-model loss, a long-range identifier loss, bug localization on SWE-bench Lite and linear probes, no summary variant improves on the no-summary baseline. A test-time ablation that hides every `<sum>` token changes long-range loss by at most ± 0.004 nats, and the summaries receive under 1% of attention. Shrinking the local window from 512 to 128 tokens does not change this: the model routes distant context through its full-attention layers instead, whose attention to far tokens rises from 35% to 57%. Linear probes do recover function properties from scope-aligned `<sum>` states far better than from fixed-chunk states (e.g. 0.92 vs. 0.71 on *returns a value*), but a mean of the function's own raw hidden states does as well or better. So the probe advantage reflects *where* the summaries sit, not a learned compression. We conclude that in hybrid models whose global layers see the full context, summary tokens added only to the local layers are redundant. Removing that shortcut, by giving every layer the same local mask, more than doubles long-range loss, and the summaries still do not recover the lost context; keeping every function signature visible recovers only a small part of it, and a SepLLM-style baseline that keeps separator tokens does better than both, at a larger memory cost. All main results hold across two training seeds.

Keywords: Code language models, Context compression, Summary tokens, KV cache, Sliding-window attention, Negative result

1 INTRODUCTION

Long code contexts are expensive: KV memory grows linearly with length, and most of a file is irrelevant to the next token. Context-compression methods insert learned summary tokens that absorb a span and stand in for it [1–4]. Almost all of them place the tokens at fixed intervals. Code, however, has explicit structure. A function is a natural unit of meaning, and a summary at its end could serve later code that calls it.

We ask a narrow, controlled question: *does placing summary tokens at code-scope boundaries help, compared with placing the same number of tokens at fixed intervals, when everything else is held fixed?* We study this in a modern hybrid-attention model, Gemma 4 E2B. Its 35 layers repeat a pattern of four sliding-window layers ($W = 512$) and one full-attention layer. We change only the attention masks of the sliding layers, so a raw token sees its last W raw tokens plus every earlier `<sum>`, and each `<sum>` sees the raw tokens of its own segment. We then fine-tune with LoRA.

Our contributions are:

- A retrofit of scope-aligned summary tokens, with nesting and optional call-graph edges, onto the local layers of a pretrained hybrid model, without resizing its vocabulary or embeddings (§3).
- A controlled comparison with matched `<sum>` counts and matched KV memory. Matching matters: the naive scope variant keeps 52% more KV live than its fixed-chunk counterpart, because a long scope must stay in memory until its summary (§3.1).
- Diagnostics that test whether the summaries are *used*: test-time hiding, attention shares and loss by position in a scope. They show that the summaries are not used, and identify the full-attention layers as the reason (§5).
- Probe controls showing that the advantage of scope-aligned summaries on per-function probes is explained by their alignment, not by compression.
- Tests with the global shortcut removed, two training seeds, a signature-keeping variant and a SepLLM-style baseline, placed on a memory–quality trade-off: at our budget, long-range quality follows how many raw tokens are kept, not the learned summaries (§6).

2 RELATED WORK

Summary and gist tokens. Gist tokens [1], AutoCompressors [2], ICAE [3] and Activation Beacon [4] compress context into learned tokens, mostly at fixed intervals. Sentence-anchored gist compression [5] anchors gists at sentence boundaries, but without a matched uniform baseline. Gist-token sparse attention [6] and adaptive semantic trees [7] organize summaries hierarchically. SepLLM [8] keeps separator tokens as implicit summaries. Kwai Summary Attention [9] builds a hybrid stack of summary and full-attention layers, with fixed chunks, trained from scratch

or with continued pretraining on far more tokens than we use.

Compression for code. CodeComp [10] compresses the KV cache using a code property graph, without training. Yang et al. [11] report that compressors hurt repository-level issue fixing. We are not aware of prior work that places *learned* summaries at code-scope boundaries, nests them along the scope tree, compares boundary and uniform placement at matched count and memory, or links summaries along call edges.

Attention sinks. Xiao et al. [12] show that models park attention on a few tokens, such as the first token. Our diagnostics test whether <sum> attention reflects information transfer or sink behaviour.

3 METHOD

Segments. A logical-line lexer and an indentation stack split a Python window into *scopes*: functions and classes (with their decorators), plus *module runs* between top-level definitions. Each scope's raw tokens are the tokens directly inside it. Tokens of nested scopes belong to the nested scope, and the scope's header (e.g. the `def` line) is marked. One <sum> is inserted right after the last raw token of each segment. Each variant defines segments differently (Table 1). Figure 1 shows this on a small example.

Masks. Global (full-attention) layers stay causal. In the sliding layers, for variants B, C, F and G, a raw query sees BOS, the last W raw tokens (counted in raw tokens) and every earlier <sum>. A <sum> query sees BOS, the raw tokens of its own segment and the <sum>s of its direct children. D and E replace the W-token window with the query's own scope, the headers of enclosing scopes and the last $w = 64$ tokens. E also lets a <sum> attend to the <sum>s of functions it calls (call edges found by regular expressions).

Formally, let $\kappa(j) \in \{\text{BOS}, \text{RAW}, \text{SUM}\}$ be the kind of position j, $r(j)$ its index among raw tokens, $s(j)$ its segment, and $\pi(g)$ the parent segment of g. In a sliding layer of variants B, C, F and G, a raw query q may attend to key $k \le q$ when

$$\begin{aligned} &\kappa(k) = \text{BOS} \ \vee \ \kappa(k) = \text{SUM} \\ &\vee \ \big(\kappa(k) = \text{RAW} \wedge r(q) - r(k) < W\big), \end{aligned} \tag{1}$$

and a <sum> query q may attend to $k \le q$ when

$$\begin{aligned} &\kappa(k) = \text{BOS} \ \vee \ \big(\kappa(k) = \text{RAW} \wedge s(k) = s(q)\big) \\ &\vee \ \big(\kappa(k) = \text{SUM} \wedge \pi(s(k)) = s(q)\big). \end{aligned} \tag{2}$$

Full-attention layers use the plain causal mask ($k \le q$) unless `restrict_full` is set, in which case they use (1)–(2) as well. Every position also attends to itself, and padding is never attended. Figure 2 shows the sliding mask for the example of Figure 1, and Figure 3 shows where these masks sit in the model.

Retrofit. Gemma 4 reserves thousands of unused tokens, so we reuse <unused0> as <sum>. Its input embedding is a trainable vector initialised to the mean embedding row, and its per-layer-embedding row is reset to the mean. We pass per-layer-type masks to the decoder as a dictionary, so the library's own sliding-window mask is replaced only in the sliding layers.

3.1 Fair comparison

B fixes the *count* of summary tokens. Its K is chosen so that the total number of fixed-chunk <sum>s over the training set matches C's (7.09 per window for both). Count matching is not enough, because the two variants differ in KV memory. In C, the raw tokens of a 2,000-token class body must stay in the cache until the class's <sum> has attended to them. The measured mean live KV is 672 entries for C and 441 for B, and the peak is 1,193 vs. 521. F removes this advantage. It cuts any scope whose span exceeds W into consecutive *parts* at logical-line starts. Each part gets a <sum> and becomes a child of the scope, and the scope's header goes into its first part. With $W = 512$ on the training windows, F's live KV matches B's (mean 289 vs. 288, peak 517 vs. 516).[1] G re-fits K so that fixed chunks match F's count (57,820 vs. 59,900 <sum>s over the training set). Figure 4 shows the memory of all variants on the evaluation windows.

4 EXPERIMENTAL SETUP

Data. Python files from The Stack [13] (`the-stack-smol-xl`). We exclude the four evaluation repositories (fastapi, requests, rich, httpx) by name, path and signature, and the twelve SWE-bench Lite repositories by exact name. Files are cut into windows of at most 1,791 raw tokens at top-level boundaries, leaving room for summaries in a 2,048-token sequence. This gives 6,967 training windows (10M tokens).

Training. LoRA [14] with rank 16 and $\alpha = 32$ on all attention and MLP projections of the text decoder, with learning rate 2×10^{-4} (10^{-3} for the <sum> embedding). Effective batch 16, 130 steps (about 3M tokens), 13 warm-up steps, float16 on one T4 GPU, about 2.7 hours per variant, two seeds (1234 and 2) for every variant in Table 4. Table 2 lists the full setup, and Figure 5 shows the training loss of the main runs.

Evaluation. (i) Heldout LM loss on 99,657 tokens. (ii) Long-range identifier loss on 206 windows from the four held-out repositories: the loss on the 2,192 identifier tokens whose definition lies more than 512 raw tokens earlier *and* in a different scope. The target set is identical for all variants. (iii) Bug localization on 19 SWE-bench Lite tasks [15]: every function in the patched files is ranked by the log-likelihood of its name after the file and the issue text. (iv) Logistic-regression probes on the last-layer <sum> state of each function (1,191 train, 347 test functions, split by file) for *returns a value*, *number of arguments* (4 buckets) and

[1] The F evaluation in Table 3 was run before the header fix and reports peak 776; loss and probes are unaffected by where headers are kept in memory.

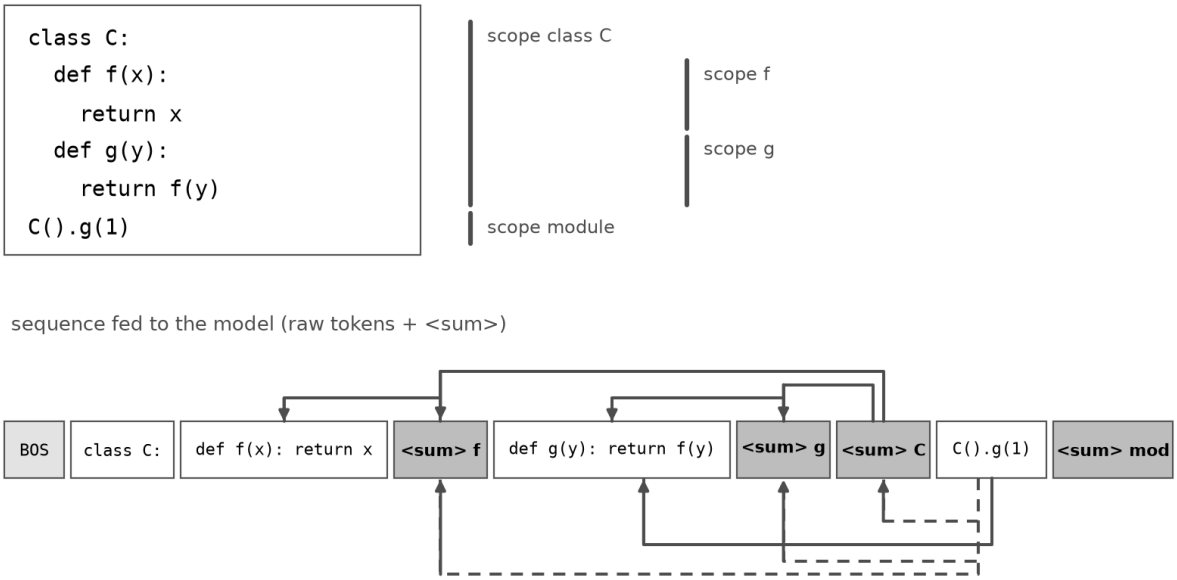


Fig. 1: *Inserting summary tokens. A scope tracker splits the code into nested scopes, and a* `<sum>` *token is inserted after the last raw token of each scope. Above the sequence: a* `<sum>` *query attends to its scope's raw tokens and to the* `<sum>`*s of nested scopes. Below: a raw query in a sliding layer attends to the last W raw tokens (solid) and to every earlier* `<sum>` *(dashed). Variant F splits a long scope into parts of at most W tokens, each with its own* `<sum>`*; B and G place the same number of* `<sum>`*s every K raw tokens instead.*

Table 1: *Variants. All use identical windows, data order and hyper-parameters. Two options change the masks:* `restrict_full` *gives the full-attention layers the same mask as the sliding layers (otherwise they see every earlier token), and* `keep_signatures` *lets raw queries also see every earlier* `def`*/*`class` *line. Separators (S) are whitespace tokens and* `. , ? ! ; :`*.*

	Segments	Sliding-layer raw query sees
A	none	last W positions (model default)
B	fixed chunks, $K = 223$ (count-matched to C)	last W raw + all earlier `<sum>`
C	scopes	last W raw + all earlier `<sum>`
D	scopes	own scope + enclosing headers + last 64 + all `<sum>`
E	scopes, + callee edges between `<sum>`	as D
F	scopes, long scopes split into parts $\leq W$	as C
G	fixed chunks, $K = 180$ (count-matched to F)	as B
S	none (SepLLM-style baseline)	last W raw + every earlier separator

Table 2: *Training setup, identical for every variant.*

Setting	Value
Base model	Gemma 4 E2B (28 sliding + 7 full)
Sliding window W	512 (128 in the stress test)
Adapter	LoRA, $r = 16$, $\alpha = 32$, dropout 0.05
LoRA targets	q, k, v, o, gate, up, down
`<sum>` token	reused `<unused0>`, trainable embedding
Learning rate	2×10^{-4} (LoRA), 10^{-3} (`<sum>`)
Schedule	13 warm-up steps, then decay to 0
Batch	1 window $\times$ 16 accumulation steps
Steps / tokens	130 / about 3M raw tokens
Sequence length	2,048 (at most 1,791 raw + `<sum>`)
Precision / hardware	float16 / one NVIDIA T4 (Kaggle)
Seeds	1234; also 2 (Table 4)

name reused later. For chunk variants the probe uses the chunk containing the function's last token. Controls: the mean of the function's raw hidden states (*raw-mean*), and the subset of functions that straddle a chunk boundary. (v) Diagnostics: loss with raw queries blocked from all `<sum>` keys (*hide*), attention mass on `<sum>`, BOS and far raw tokens (eager attention, 16 windows), and loss by offset from scope start.

5 RESULTS

No variant beats the baseline. Table 3 and Figure 6a show heldout and long-range losses within 0.004 nats of each other for A, B, C, F and G. D and E are 0.03 nats worse: their raw queries lose the most recent 512 tokens outside their own scope, and the summaries do not make up for it. Bug localization does not separate the variants on 19 tasks. A second seed (Table 4) moves A's long-range loss by 0.007 (0.318 to 0.325), more than the gaps between A, F

Table 3: *Main results at $W = 512$. Lower loss is better. Differences below about 0.01 nats are within run-to-run noise (seed 1; Table 4 adds seed 2).*

	Heldout loss	All-token loss (eval)	Long-range loss	Live KV mean / peak	Loc. top-1 / top-5 (of 19)
A baseline	**1.989**	**0.722**	0.318	437 / 512	4 / 10
B chunks (K=223)	1.992	0.724	**0.317**	441 / 521	3 / 9
C scopes	1.991	0.724	0.324	672 / 1193	3 / 11
D scope windows	2.020	0.750	0.352	519 / 1036	4 / 9
E + call edges	2.021	0.749	0.354	519 / 1036	4 / 9
F scopes, parts $\leq W$	1.991	0.724	0.321	536 / 776	3 / 11
G chunks (K=180)	1.993	0.726	0.320	442 / 522	3 / 9

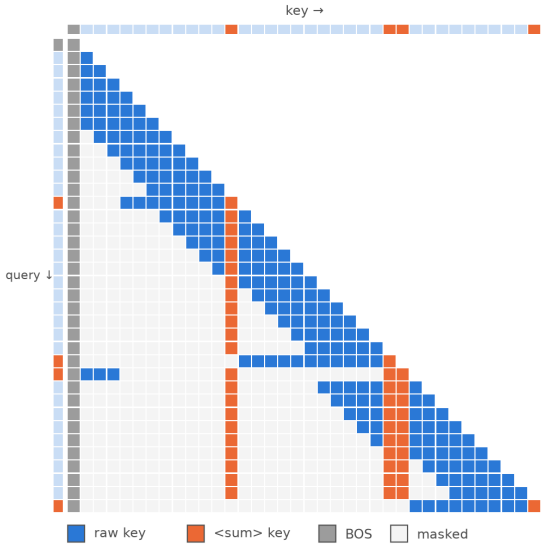


Fig. 2: *The sliding-layer mask our code builds for the example of Figure 1 (variant C, $W = 6$; row = query, column = key). Raw rows see a band of the last W raw tokens plus every earlier `<sum>`; `<sum>` rows see their own scope's raw tokens and the `<sum>` of nested scopes.*

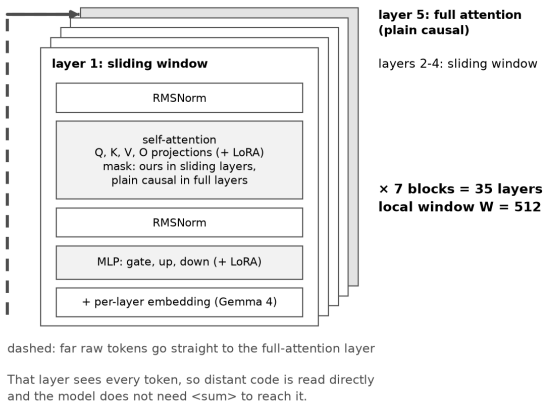


Fig. 3: *Where the masks apply. Gemma 4 E2B stacks 7 blocks of five layers; the front layer shows what each layer contains. Our masks replace only the attention mask of the four sliding-window layers. The fifth layer uses full attention and still sees every token, so far code can reach it directly (dashed path).*

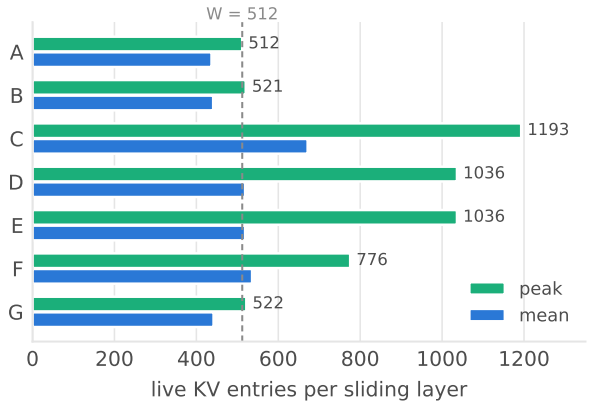


Fig. 4: *Live KV entries per sliding layer on the evaluation windows (mean and peak over positions). C, D and E keep long scopes in memory until their `<sum>`; B and G stay at about W. F was measured before the header fix (peak 776); after it, F's peak matches B's on the training windows.*

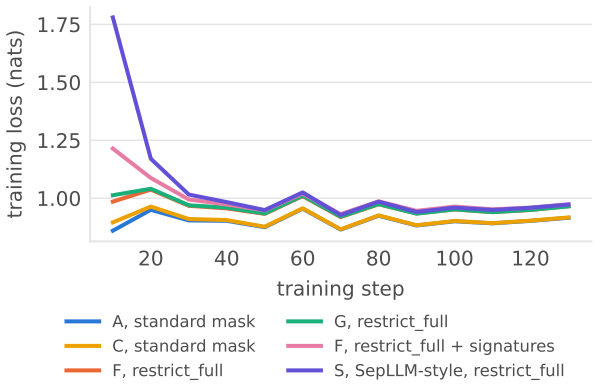


Fig. 5: *Training loss of the main runs (logged every 10 steps). Runs with the same mask track each other because all variants see the same windows in the same order; `restrict_full` raises the loss throughout training.*

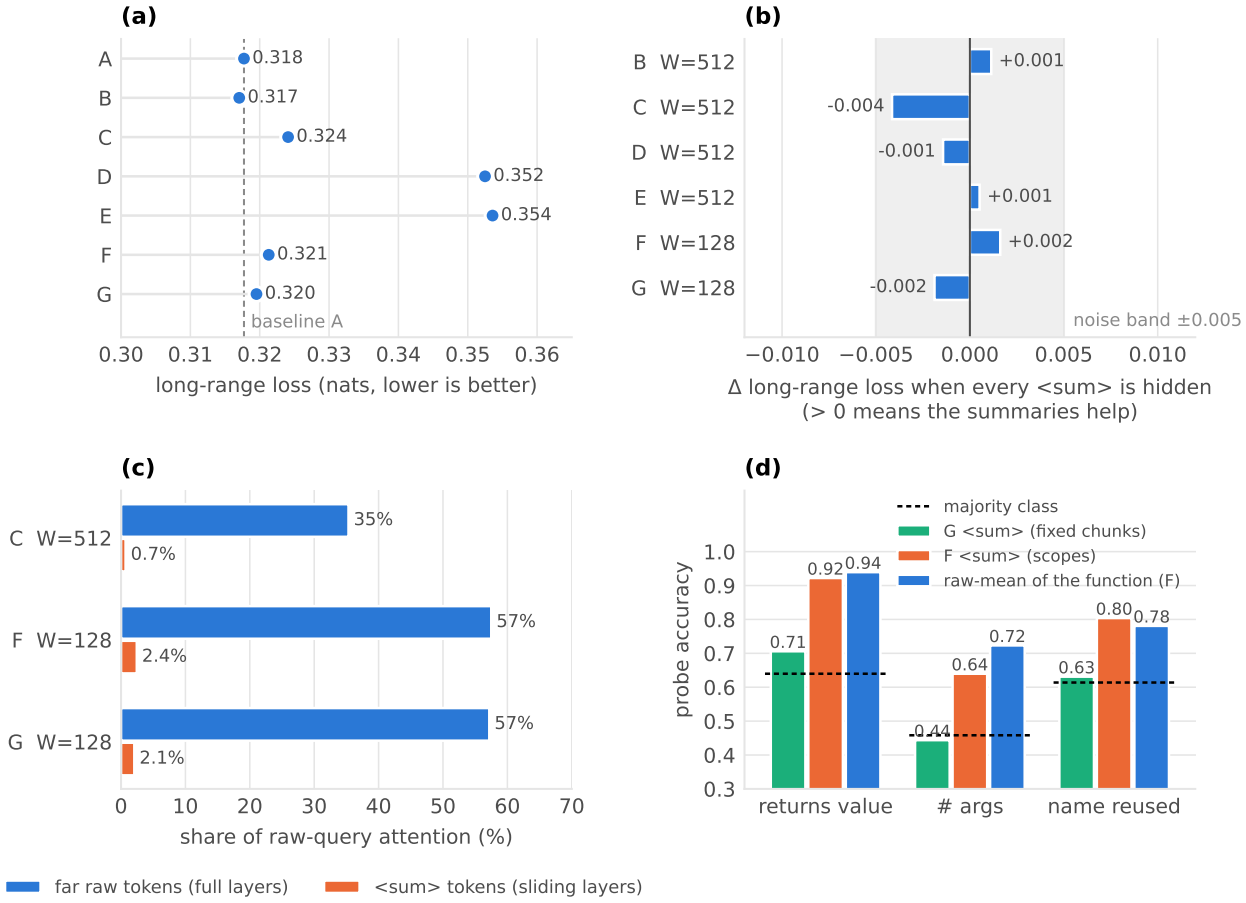


Fig. 6: *Summary of the results. (a) Long-range loss: no variant beats the baseline A. (b) Hiding every* `<sum>` *at test time changes long-range loss by less than the noise band, in both directions. (c) The full-attention layers carry distant context (35% of their attention at $W = 512$, 57% at $W = 128$), while the* `<sum>` *tokens get a few percent. (d) Scope-aligned* `<sum>` *states (F) encode function properties far better than fixed-chunk ones (G), but no better than the mean of the function's raw hidden states.*

and G (0.320–0.322 in seed 2), so with the standard mask there is no measurable difference between them.

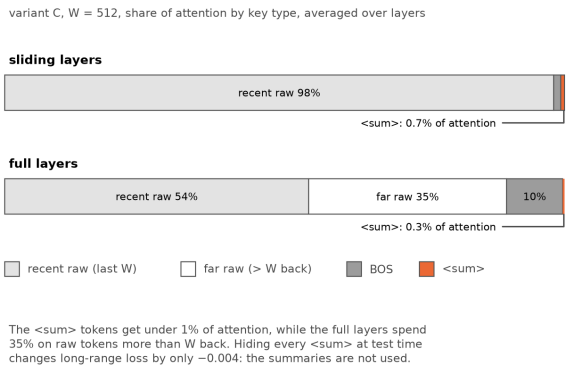


Fig. 7: *Where a raw query's attention goes, measured on variant C at $W = 512$.* `<sum>` *keys receive 0.7% of attention in the sliding layers and 0.3% in the full-attention layers, while the full layers spend 35% on raw tokens more than W back.*

The summaries are not used. Hiding all `<sum>` keys at test time changes long-range loss by -0.004 to $+0.001$ nats at $W = 512$ (Table 5, Figure 6b). In C, hiding them slightly *improves* the loss. The summaries receive under 1% of attention (Figure 7), about a tenth of what BOS receives in the full-attention layers (10%). Loss by offset from the start of a scope, where a summary of the preceding code should help most, is the same for A, B and C (0.977, 0.979 and 0.982 nats on the first 16 tokens; Table 6).

Per layer (Figure 8), the little attention the summaries receive is concentrated in the first layers (up to 11% in layer 0 at $W = 128$) and stays below about 2% in the later layers, where the model forms its predictions; closing the shortcut does not move it there. Raw tokens kept beyond the window are used very differently: signatures receive 5–22% and separators 10–43% of attention in many layers.

Why: the global layers already see everything. In the full-attention layers, 35% of the attention of raw queries goes to raw tokens more than 512 positions back. Distant code is therefore available without the summaries. To test whether a narrower local window would force the model to use them, we retrained F and G with $W = 128$ (G with $K =$

Table 4: *Two training seeds (seed 2 changes the LoRA initialisation and the data order). Δ is the change in long-range loss when every <sum> is hidden at test time; it stays at or below zero in all 13 diagnosed runs. Seed 1 of F (standard mask) was trained before long scopes moved their header into the first part and diagnosed after, so its segments differ slightly from training. S has no <sum> to hide.*

		Long-range loss			Δ when <sum> hidden	
Mask	Variant	seed 1	seed 2	mean	seed 1	seed 2
standard	A	0.318	0.325	0.321	–	–
standard	F	0.321	0.321	0.321	−0.003	−0.0003
standard	G	0.320	0.322	0.321	−0.0001	−0.004
restrict_full	F	0.735	0.720	0.728	−0.008	−0.014
restrict_full	G	0.694	0.713	0.704	−0.004	−0.001
restrict_full	F + signatures	0.691	0.694	0.693	−0.001	−0.002
restrict_full	S (SepLLM-style)	0.632	0.630	0.631	–	–

Table 5: *Are the summaries used? Δ is the change in loss when every raw query is blocked from every <sum> at test time. A positive value means the summaries help. Far-raw: attention from raw queries to raw keys more than W tokens back, which only the full-attention layers can see.*

		<sum> attention	Far-raw attention	Δ loss when hidden	
W	Var.	sliding / full	full layers	long-range	all
512	B	1.0% / 0.6%	35%	+0.001	−0.001
512	C	0.7% / 0.3%	35%	−0.004	−0.001
512	D	0.8% / 0.3%	33%	−0.001	−0.001
512	E	0.7% / 0.4%	34%	+0.001	−0.000
128	F	2.4% / 0.6%	57%	+0.002	−0.000
128	G (*K*=90)	2.1% / 0.9%	57%	−0.002	−0.000

Table 6: *All-token loss by position within the enclosing scope, $W = 512$. Summaries give no gain at scope starts, where earlier code should matter most.*

		offset from scope start (tokens)			
Mask	Variant	0–15	16–63	64–255	256+
standard	A	0.977	0.798	0.688	0.645
standard	B	0.979	0.800	0.690	0.647
standard	C	0.982	0.800	0.689	0.647
restrict_full	F	1.027	0.838	0.718	0.684
restrict_full	G	1.021	0.835	0.719	0.683
restrict_full + signatures	F	1.023	0.852	0.739	0.704
restrict_full	S (SepLLM-style)	1.025	0.844	0.728	0.697

90 to match F's count). The loss cost is small (heldout 2.004 and 2.008 vs. 1.991). Attention to <sum> rises to 2.4%, but hiding the summaries still changes long-range loss by only ±0.002. The model instead shifts distant context to the full-attention layers, whose far-raw share rises (Figure 6c) from 35% to 57%.

What the summary states contain. Probes on scope-aligned <sum> states predict function properties well above the majority class, and well above fixed-chunk states (Table 7, Figure 6d): *returns a value* reaches 0.92 for F and 0.71 for G. The gap is largest for functions that straddle a chunk boundary, where no single chunk summary covers the function (0.906 vs. 0.680). However, the mean of the function's raw hidden states matches or beats the F <sum> (0.939 vs. 0.922; 0.723 vs. 0.640 on argument count). A <sum> at the end of a function attends to that function's tokens, so its state is a pooled view of the function. The probes show that this view is aligned with the unit being probed, not that training produced a compressed representation the rest of the model relies on. The hiding results show that it does not rely on it.

5.1 Cost: tokens and KV memory

Summary tokens are added, never substituted, so the sequence grows: by 4.9 (B, C), 5.8 (G) or 6.0 (F) <sum>s per 1,000 raw tokens, measured on all 10M training tokens, an overhead of 0.5–0.6%. Whether anything is saved depends on the full-attention layers (Table 8, Figure 9). We project the fp16 KV cache per sequence, the keys a streaming implementation must keep, from the masks and Gemma 4 E2B's cache layout (12 sliding and 3 full-attention layers own KV, one KV head each; the other 20 layers share them). With the standard mask the full layers keep every token, so summaries save nothing and cost slightly more than the

Table 7: *Probe accuracy on held-out functions. Scope-aligned* `<sum>` *states (C–F) carry far more information about their function than fixed-chunk states (B, G), but no more than the mean of the function's raw hidden states. On the 181 functions that straddle a chunk boundary,* returns a value *is 0.906 for F, 0.680 for G and 0.928 for raw-mean.*

	`<sum>` state						raw-mean	
Task (majority)	B	C	D	E	F	G	F	G
returns value (0.640)	0.709	0.905	0.885	**0.939**	0.922	0.706	**0.939**	0.934
# args (0.458)	0.401	0.614	0.611	0.617	0.640	0.444	**0.723**	0.709
name reused (0.614)	0.602	0.790	0.795	**0.813**	0.804	0.631	0.781	0.767

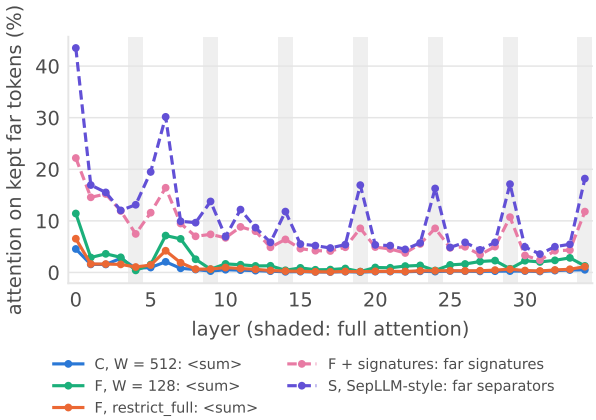


Fig. 8: *Per layer (shaded: full attention), the share of raw-query attention on the tokens each variant keeps beyond the window:* `<sum>` *keys (C, F) or kept raw tokens more than W back (signatures, separators). The model attends to kept raw tokens heavily (separators up to 43%, signatures up to 22%) but to the summaries mostly below 2%, even when they are the only route past W (*`restrict_full`*).*

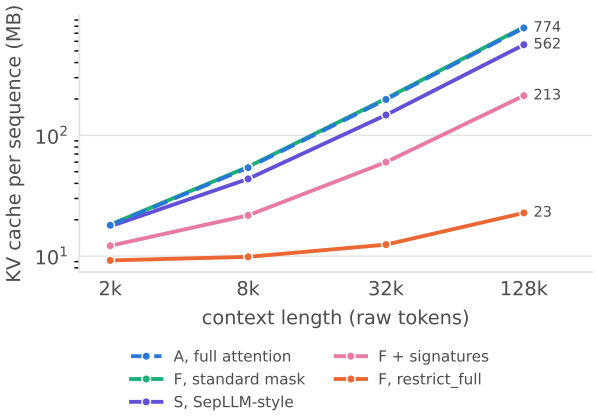


Fig. 9: *Projected KV cache per sequence against context length. With the standard mask, F overlaps the baseline A because the full-attention layers keep every token. With* `restrict_full`*, the cache grows with the number of summaries (F), plus every kept signature (F + signatures) or separator (S).*

baseline. With `restrict_full` every layer keeps only the last W raw tokens plus the summaries, so the cache grows with the number of summaries instead of the context: at 128k tokens it is 34× smaller, and a new token reads 21× fewer keys. Keeping raw tokens for good is costlier: signatures (8.2% of tokens) give 213 MB and separators (24.0%) 562 MB at 128k, and because those tokens stay visible in all 35 layers, the SepLLM-style variant reads more keys per new token than full attention (1.12M vs. 0.93M), whose sliding layers read only their 512-token window. This saving comes from the mask; the summaries are what should recover the distant information the mask hides, which they do not (§6). In our fp16 setup the absolute saving is modest (0.77 GB of KV against about 9 GB of weights), because the model's cache is already small; it would matter more for longer contexts, many concurrent sequences, or models with larger caches. We have not measured memory on a quantized or on-device deployment. These are projections: our implementation computes dense attention and then masks it, so the measured peak memory is the same for every variant (9.05 GB at 2k).

6 DISCUSSION

Implications. Hybrid stacks that interleave local and global attention are now common. In such models, summary tokens added only to the local layers compete with a global path that sees every raw token. Under a short fine-tune, the model takes the global path. Positive results for summary tokens in hybrid models (e.g. 9) build the summaries into the architecture and train on orders of magnitude more tokens. Our result suggests that the global-layer path, not the placement of the summaries, decides whether they matter.

Closing the shortcut. We gave the full-attention layers the same mask as the sliding layers (`restrict_full`, $W = 512$), so raw tokens more than W back can only be reached through `<sum>`, and retrained F and G for 130 steps (Table 9). Far-raw attention in the full layers drops from 35% to zero, and long-range loss more than doubles (0.32 to 0.69–0.74 in both seeds): distant code matters. The summaries do not recover it. Hiding them still lowers long-range loss slightly in both seeds (-0.008 and -0.014 for F, -0.004 and -0.001 for G), so even as the only route past W they carry nothing the model uses at this training budget, and the hidden condition, close to a StreamingLLM-style window with a sink [12], does as well. Fixed chunks (G) are at least as good as scope parts (F) in both seeds (0.694 vs. 0.735 and 0.713 vs. 0.720), but the margin depends on

Table 8: *Projected cost per sequence for Gemma 4 E2B (fp16). Keys read are summed over all 35 layers. G is within 2% of F throughout. The savings come from the mask; with* `restrict_full` *they cost long-range quality that the summaries do not recover (Table 9). Raw tokens kept for good (signatures, separators) are kept in every layer, so S reads more keys per new token than full attention.*

	KV cache per sequence (MB)				keys read per
Setting	2k	8k	32k	128k	new token, 128k
A, full attention	18.0	54.0	198	774	932k
F, standard mask	18.2	54.9	201	788	959k
F, `restrict_full`	9.2	9.9	12.4	22.8	45k
F + signatures, `restrict_full`	12.2	21.7	60.0	213	424k
S (SepLLM-style), `restrict_full`	17.6	43.6	147	562	1,119k

the seed.

Keeping signatures. A single <sum> vector may hold what a function does, but code that calls the function needs its exact name and parameters, and compressed tokens are known to be poor at exact copying. We therefore add an option (`keep_signatures`) under which raw queries, in every layer when combined with `restrict_full`, also see every earlier `def` or `class` line (including decorators), while function bodies stay reachable only through <sum>. Signature lines are 8.2% of the raw tokens in our training data, so every layer's cache grows by about 88 entries per 1,000 tokens (signatures plus summaries). Because the sliding layers must now keep the signatures too, the projected cache at 128k tokens is about 213 MB, 3.6× smaller than the full cache (774 MB) but larger than with summaries alone (23 MB). We trained F with `restrict_full` and `keep_signatures` for 130 steps. The model does read the signatures (7.4% of attention in the sliding layers, 8.7% in the full layers), and long-range loss improves to 0.691 (0.694 with seed 2; mean 0.693, against 0.728 for F and 0.704 for G). The gain is small and consistent, remains more than twice the unrestricted 0.32, and comes with a slightly worse all-token loss (0.777 vs. 0.761). Hiding the <sum>s changes long-range loss by −0.0005, so the small gain comes from the signatures, not the summaries. Exact signatures are not what the lost distant context mainly provides.

A SepLLM-style baseline. Variant S has no <sum> tokens: under `restrict_full`, every layer sees BOS, the last W raw tokens and every earlier separator token (whitespace and `. , ? ! ; :`), the separator set of SepLLM [8]. It reaches a long-range loss of 0.632 and 0.630 in two seeds, better than every <sum> variant and the signature variant. It is not memory-matched: separators are 24.0% of the raw tokens in our training data, against 8.2% for signatures and 0.6% for summaries, so its projected cache at 128k tokens is 562 MB, against 213 MB and 23 MB (774 MB for full attention). Figure 10 shows the resulting trade-off: keeping more raw tokens buys long-range accuracy, and at this training budget a learned <sum> buys none. Whether S still wins at an equal memory budget is open.

Threats to validity. *Internal:* all variants share data, order, seed and hyper-parameters, so differences come from the masks and segments; but with two seeds, gaps below about

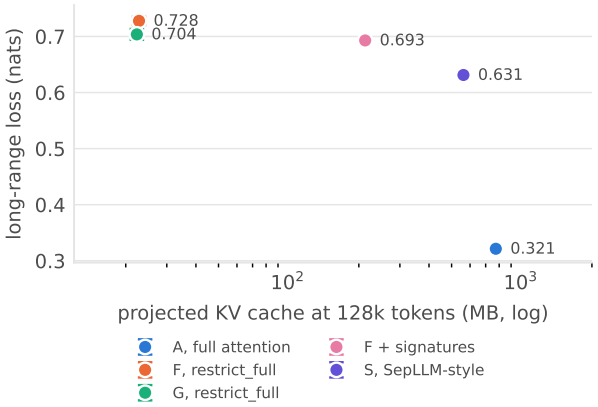


Fig. 10: *Long-range loss against the projected KV cache at 128k tokens, mean over two seeds (bars: range). Under* `restrict_full`*, quality improves only with the number of raw tokens kept (signatures, separators); the summaries alone stay at the cheapest and worst point.*

0.01 nats are not significant, and our conclusions rest only on the larger, consistent effects (near-zero hiding effects across five settings, and the doubling of long-range loss under `restrict_full`). *Construct:* long-range loss scores identifier tokens defined more than 512 tokens earlier in another scope; it measures access to distant definitions, not end-to-end code generation. *External:* we study one model (Gemma 4 E2B), one language (Python) and one corpus; models with different attention layouts or much longer training may behave differently.

Limitations. The training is short: 3M tokens, rank-16 LoRA, two seeds per variant. The probe and localization sets are small. The <sum> embedding starts from a reused token. Long-range loss uses one definition of long range (more than 512 tokens, across scopes). Longer training could teach the model to use the summaries even with the global path open. Our results show only that it does not do so at this budget, where the main effect of the method would be expected to appear first.

7 CONCLUSION

Scope-aligned summary tokens retrofitted onto Gemma 4's sliding-window layers do not improve language

Table 9: *Closing the global shortcut ($W = 512$ in every layer). Long-range loss more than doubles, and hiding the summaries still does not hurt. Keeping every* `def`/`class` *line visible (last row) helps slightly, through the signatures, not the summaries.*

	Heldout loss	Long-range loss	Long-range, <sum> hidden	Δ hide	<sum> attention sliding / full
F, standard mask	1.991	0.321	–	–	–
F, `restrict_full`	2.047	0.735	0.727	−0.008	1.0% / 0.6%
G, `restrict_full`	2.048	0.694	0.690	−0.004	1.5% / 0.7%
F, `restrict_full` + signatures	2.055	0.691	0.691	−0.0005	0.6% / 0.6%
S (SepLLM-style), `restrict_full`	2.047	0.632	–	–	–

modelling, long-range identifier prediction or bug localization over a no-summary baseline, even when the comparison is matched for count and memory, and even when the local window is cut to 128 tokens. The model ignores the summaries, because its full-attention layers already reach distant code. Scope-aligned summary states are much more informative than fixed-chunk ones, but only as informative as pooling the scope's own tokens. Removing the global shortcut does not change this: with every layer restricted to the last 512 tokens, distant code matters (long-range loss more than doubles), but the summaries still carry nothing the model uses. Keeping every function signature visible recovers only a small part of the loss, and the summaries add nothing on top of it. A SepLLM-style baseline that keeps every separator token does better than all of them, at 2.6 times the memory of the signature variant: at this budget, long-range quality follows how many raw tokens are kept, not learned summaries. Making summary tokens useful for code would likely need much longer training than our 3M tokens.

REFERENCES

[1] J. Mu, X. L. Li, and N. Goodman. Learning to compress prompts with gist tokens. *NeurIPS*, 2023. arXiv:2304.08467.

[2] A. Chevalier, A. Wettig, A. Ajith, and D. Chen. Adapting language models to compress contexts. *EMNLP*, 2023. arXiv:2305.14788.

[3] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei. In-context autoencoder for context compression in a large language model. *ICLR*, 2024. arXiv:2307.06945.

[4] P. Zhang et al. Long context compression with activation beacon. arXiv:2401.03462, 2024.

[5] D. Tarasov, E. Goncharova, and A. Kuznetsov. Sentence-anchored gist compression for long-context LLMs. arXiv:2511.08128, 2025.

[6] Y. Mao, M. Y. Li, and E. B. Fox. Simplified sparse attention via gist tokens. arXiv:2604.20920, 2026.

[7] Y. Li, S. Chen, Y. Li, Y. Chen, H.-T. Zheng, H. Wang, W. Jiang, and P. S. Yu. AdmTree: Compressing lengthy context with adaptive semantic trees. arXiv:2512.04550, 2025.

[8] G. Chen et al. SepLLM: Accelerate large language models by compressing one segment into one separator. arXiv:2412.12094, 2024.

[9] C. Chu, G. Zhou, G. Zhang, H. Li, H. Peng, et al. Kwai Summary Attention technical report. arXiv:2604.24432, 2026.

[10] Q. Chen, J. Xiong, C. Zhao, S. Yang, and N. Wong. CodeComp: Structural KV cache compression for agentic coding. arXiv:2604.10235, 2026.

[11] X. Yang, R. Chatterjee, H. Udupi, V. Ramavarapu, and C. Zhai. Skip the compressor: Coding agents fix more issues when they pick whole files. arXiv:2610.21073, 2026.

[12] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. *ICLR*, 2024. arXiv:2309.17453.

[13] D. Kocetkov et al. The Stack: 3 TB of permissively licensed source code. arXiv:2211.15533, 2022.

[14] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022. arXiv:2106.09685.

[15] C. E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024. arXiv:2310.06770.